



دانشگاه تهران

دانشکده مهندسی برق و کامپیوتر

درس شبکه‌های عصبی و یادگیری عمیق
پاسخ تمرین شماره دو

دانشجویان:

سوال یک: سید احمد رکنی حسینی ۸۱۰۱۰۴۳۰۶

سوال دو: علی ممتحن ۸۱۰۱۰۴۳۲۱

فهرست مطالب

فهرست تصاویر ۳

فهرست جداول ۴

۱ تشخیص بیماری های برگ برنج با شبکه‌های عصبی ۵

۱—۱ آماده‌سازی مجموعه داده ۵

۱—۲ پیش پردازش داده‌ها ۶

۱—۳ مدل AlexNet ۸

۱—۳—۱ پیاده سازی ۸

۱—۳—۲ نتایج و ارزیابی ۹

۱—۴ مدل پیشنهادی مقاله ۱۲

۱—۴—۱ پیاده سازی ۱۲

۱—۴—۲ نتایج و ارزیابی ۱۲

۲ طبقه بندی صحنه در تصاویر سنجش از دور ۱۶

۲—۱ آماده سازی مجموعه داده ۱۶

۲—۲ پیش پردازش داده ها ۱۶

۲—۳ پیاده سازی ۱۷

۲—۴ ارزیابی ۱۸

۲—۵ تحلیل و بهینه سازی ۲۱

۲—۵—۱ تاثیر داده افزایی ۲۱

۲—۵—۲ نقش بهینه ساز ۲۲

۲—۵—۳ بهینه سازی توسط fine-tuning ۲۳

فهرست تصاویر

۱ توزیع کلاس‌های مختلف داده‌ها ۶
۲ تعداد انواع داده‌ها برای آموزش، تست و ولیدیشن ۶
۳ نمونه داده‌های آموزش پس از داده افزایی ۷
۴ نمودار تغییرات خطا و دقت مدل AlexNet ۹
۵ ماتریس درهم ریختگی مدل AlexNet ۱۱
۶ نمودار تغییرات خطا و دقت مدل یادگیری انتقالی ۱۳
۷ ماتریس درهم ریختگی مدل یادگیری انتقالی ۱۴
۸ توزیع کلاس‌های مختلف داده‌ها ۱۶
۹ نمونه تغیر تصویر برای افزایش داده ۱۷
۱۰ معماری بخش کلاس بندی مدل ارائه شده در مقاله ۱۷
۱۱ نمودار خطا و دقت در آموزش و تست ۱۹
۱۲ ماتریس درهم ریختگی مدل mlp cnn برای داده تست ۲۰
۱۳ نمودار دقت برای تست و آموزش در مدل fn cnn ۲۱
۱۴ نمودار دقت و خطا برای مدل بدون افزایش داده ۲۲
۱۵ توزیع کلاس‌های مختلف داده‌ها ۲۲
۱۶ نمودار خطا و دقت مدل فاین تیون هنگام آموزش ۲۳

فهرست جداول

۱ مدل Summary ۸
۲ پارامترهای مدل ۹
۳ معیارهای ارزیابی مدل AlexNet ۱۰
۴ جمع بندی مدل با یادگیری انتقالی ۱۲
۵ پارامترهای مدل ۱۳
۶ معیارهای ارزیابی مدل یادگیری انتقالی ۱۴
۷ دقت مدل به ازای هایپرپارامتر های متفاوت ۱۸
۸ دقت مدل به ازای هایپرپارامتر های متفاوت در فاین تیونینگ ۲۳

۱ تشخیص بیماری های برگ برنج با شبکه‌های عصبی

در این سوال به مقایسه دو روش در تشخیص بیماری های گیاه برنج می پردازیم.

روش اول روشی است که از معماری AlexNet برای این کار استفاده می کند و روش دوم روشی است که در مقاله identification disease plant image-based for learning transfer deep Using آمده است.

کدهای این سوال در فایل $Q۱.ipynb$ آمده است

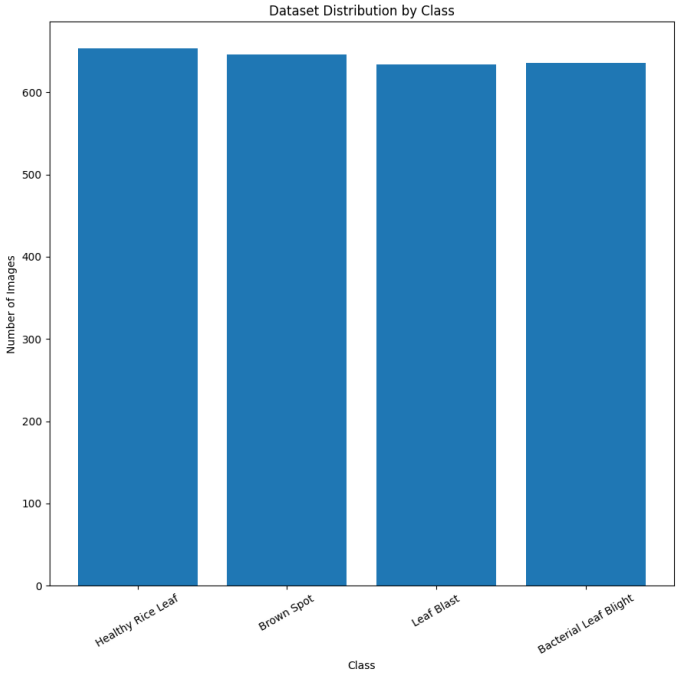
۱–۱ آماده‌سازی مجموعه داده

همانطوری که در صورت سوال آمده است، دیتا ها را به نسبت ۱۵-۱۵-۷۰ برای آموزش، تست و ولیدیشن تقسیم کردیم.

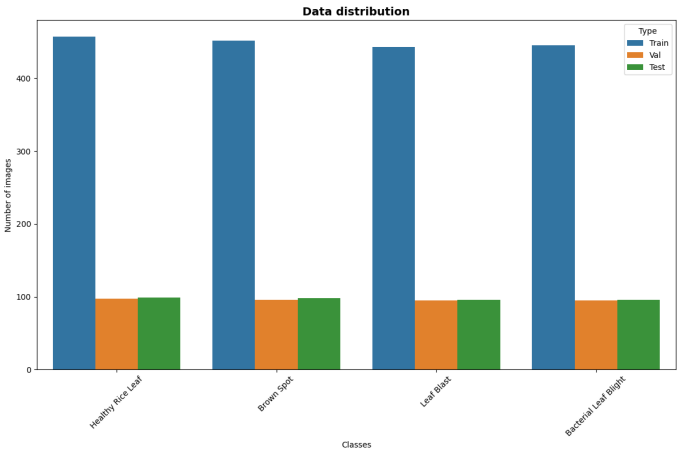
در این بخش سه زیرمجموعه از عکس‌های این سوال استفاده شده است. این سه زیرمجموعه عبارتند از

- Leaf Rice Healthy
- Spot Brown
- Blast Leaf
- Blight Leaf Bacterial

نمودار تعداد کل داده‌های هر بخش در شکل ۱ و نمودار تفکیکی آن برای هر بخش آموزش، تست و ولیدیشن در شکلی ۲ آمده است.



شکل ۱ : توزیع کلاس‌های مختلف داده‌ها



شکل ۲ : تعداد انواع داده‌ها برای آموزش، تست و ولیدیشن

۱–۲ پیش پردازش داده‌ها

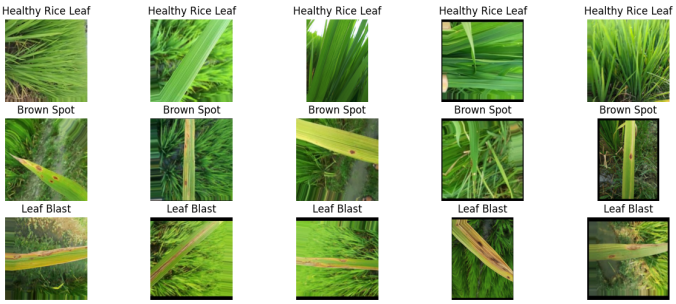
برای داده افزایی با توجه به متن مقاله، از سه روش که در مقاله ذکر هم شده اند استفاده شده است دلیل استفاده از هر یک این روش‌ها هم در روبه‌روی آن آمده است :

۱. دوران به صورت اتفاقی بین $\pm ۴۰^{o}$ به سمت راست یا چپ نسبت به نقطه وسط تصویر : باتوجه به ذات مسئله که عکس برداری از آفت های گیاه است، کاملا امکان پذیر است یک عکس به صورت صاف گرفته نشده باشد و مدل ما باید نسبت به این نوع عکس ها مقاوم باشد. همچنین خود آفت‌ها هم ممکن است در زوایای متفاوتی ایجاد بشوند و زاویه آن‌ها جزو ذات آن ها نیست.

۲. زوم کردن به صورت اتفاقی بین ۰/۹ تا ۱/۱ . : تصویر آفت‌ها می تواند از فاصله دور یا نزدیک گرفته شود.

۳. وارون سازی : همانطور که در رابطه با زاویه دهی ذکر شد، جهت آفت ها هم جزو ذات مسئله آن ها نیستند و آفت ها می توانند به هر جهتی باشند.

نمونه‌ای از داده‌ها پس از داده افزایی در شکل ۳ آمده است.



شکل ۳: نمونه داده‌های آموزش پس از داده افزایی

در ابتدا کدی توسط من برای امر داده افزایی نوشته شده بود اما به علت اینکه انجام به صورت on-the-fly نبود و روی دیسک ذخیره می شد سرعت بالایی نداشت.

پس از آن از کدی که به صورت on-the-fly موراد را اجرا می کرد استفاده شد اما شکل ۳ همچنان با استفاده از کد قبلی تولید شده است و آن کد همچنان به صورت کامت در نوت بوک موجود است.

۱–۳ مدل AlexNet

۱–۳–۱ پیاده سازی

با استفاده از نرخ یادگیری ۰/۰۰۱ مدل AlexNet پیاده سازی شده که در جدول ؟؟ summary آن امده است، مدل آموزش یافت.

جدول ۱: مدل Summary

Layer (type)	*Output Shape*	*Param #*
Conv2D (conv2d_251)	*(None, 54, 54, 96)*	*34,944*
BatchNormalization (batch_normalization_55)	*(None, 54, 54, 96)*	*384*
MaxPooling2D (max_pooling2d_114)	*(None, 26, 26, 96)*	*0*
Conv2D (conv2d_252)	*(None, 26, 26, 256)*	*614,656*
BatchNormalization (batch_normalization_56)	*(None, 26, 26, 256)*	*1,024*
MaxPooling2D (max_pooling2d_115)	*(None, 12, 12, 256)*	*0*
Conv2D (conv2d_253)	*(None, 12, 12, 384)*	*885,120*
Conv2D (conv2d_254)	*(None, 12, 12, 384)*	*1,327,488*
Conv2D (conv2d_255)	*(None, 12, 12, 256)*	*884,992*
MaxPooling2D (max_pooling2d_116)	*(None, 5, 5, 256)*	*0*
Flatten (flatten_38)	*(None, 6400)*	*0*
Dense (dense_114)	*(None, 4096)*	*26,218,496*
Dropout (dropout_76)	*(None, 4096)*	*0*
Dense (dense_115)	*(None, 4096)*	*16,781,312*
Dropout (dropout_77)	*(None, 4096)*	*0*
Dense (dense_116)	*(None, 4)*	*16,388*

برای انتخاب ابرپارامترهای آن انواع نرخ های یادگیری شامل ۰/۰۰۰۱ ,۰/۰۰۱ ,۰/۰۱ امتحان شد که بهترین آن ها در طی ۳۰ ایپاک ۰/۰۰۰۱ بود.

همچنین دو optimizer شامل SGD adam، امتحان شدند که SGD با اختلاف بهتر بود. برای Batch size هم اعداد ۲۰۰ و ۱۰۰ و ۳۲ امتحان شدند و عدد ۳۲ بهتر از دو عدد دیگر پاسخگو بود.

تمامی ابرپارامتر ها در جدول ۲ آمده اند.

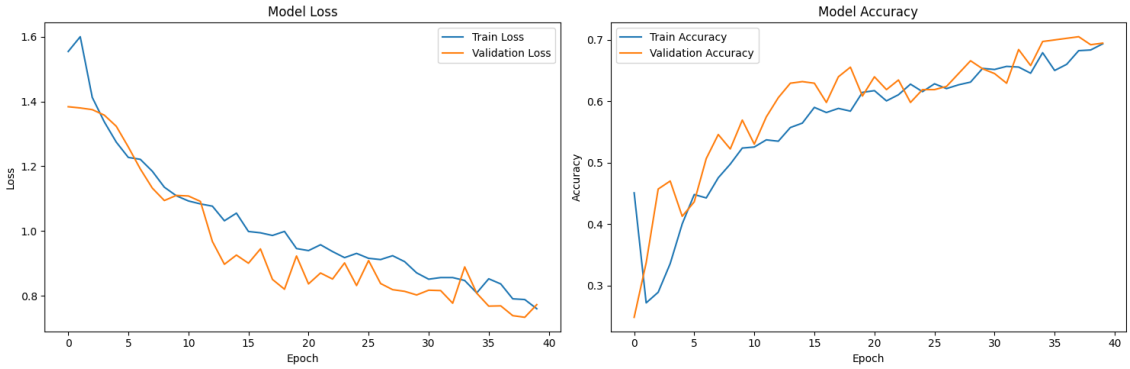
جدول ۲: پارامترهای مدل

ابرپارامتر	مقدار
Batch Size	*32*
Learning Rate	*0.0001*
Optimizer	*SGD*

۱–۳–۲ نتایج و ارزیابی

پس از اینکه به نتیجه رسیدیم که نرخ یادگیری ۰/۰۰۰۱ برای مدل مناسب است، مدل را در مجموع به اندازه ۴۰ ایپاک آموزش دادیم و نتایج زیر به دست آمد.

نمودار تغییرات خطا و دقت مدل در شکل ۴ آمده است.



شکل ۴: نمودار تغییرات خطا و دقت مدل AlexNet

همانطور که مشخص است دقت مدل در انتها به حدود ۷۰ درصد رسید.

در جدول ۳ معیارهای ارزیابی مدل آمده اند.

هر کدام از معیارهای بالا به ما در کشف یک سری ویژگی کمک می کنند.

برای فهم جدول

جدول ۳: معیارهای ارزیابی مدل AlexNet

Types	*Accuracy*	*Sensitivity*	*Specificity*
Healthy Rice Leaf	*0.9486*	*0.8182*	*0.9931*
Brown Spot	*0.8278*	*0.6837*	*0.8763*
Leaf Blast	*0.7892*	*0.4271*	*0.9078*
Bacterial Leaf Blight	*0.8483*	*0.8958*	*0.8328*

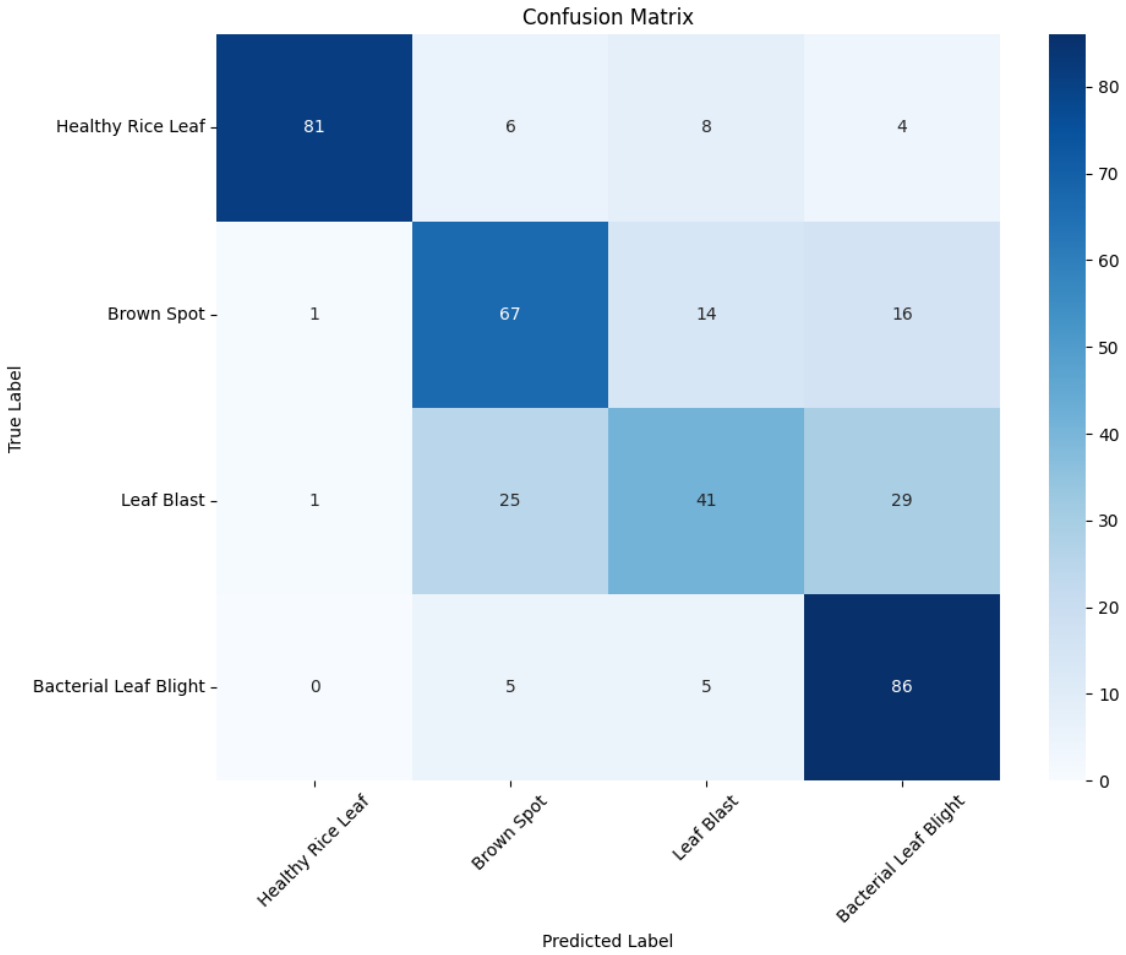
لازم است ابتدا معنی حدودی هر یک معبارهای جدول ۳ را بفهمیم :

- Accuracy : درصد نمونه‌های درست شناسایی شده به صورت کلی.
- Sensivity : زیاد بودن این عدد نشان می دهد که مدل در تشخیص نمونه های واقعی یک کلاس از غیر آن به خوبی عمل می کند.
- Specificity : درصد نمونه‌های غیر آن کلاس که به درستی به عنوان غیرکلاس شناسایی شده.

باتوجه به موارد بالا در میابیم :

- مدل بهترین عملکرد را در تشخیص برگ‌های سالم و Blight Leaf Bacterial دارد.
- مدل اغلب کلاس Blast Leaf را با کلاس‌های دیگر اشتباه می‌گیرد.
- در کل مدل در بخش Specifity خوب عمل کرده است اما در بخش Sensitivity مشکل دارد.

۵ آمده است.



شکل ۵: ماتریس درهم ریختگی مدل AlexNet

از شکل ۵ که نشان دهنده ماتریس درهم ریختگی مدل است متوجه می شویم که مدل در تشخیص Bacterial Blight Leaf و Healthy Rice Leaf به خوبی عمل کرده است.

اما در تشخیص Blast Leaf موفق نبوده است و آن را به شدت با دو کلاس Brown Spot و Bacterial Blight Leaf قاطی می کند.

۱–۴ مدل پیشنهادی مقاله

۱–۴–۱ پیاده سازی

این بخش در نوت بوک پیاده سازی شده است اما summary آن در شکل ۴ آمده است. منظور از لایه اول همان مدل VGG-۱۹ می باشد .

جدول ۴: جمع بندی مدل با یادگیری انتقالی

Layer (type)	*Output Shape*	*Param #*
functional_3 (Functional)	*(None, 28, 28, 512)*	*8,225,344*
Pool3 (MaxPooling2D)	*(None, 14, 14, 512)*	*0*
conv2d_15 (Conv2D)	*(None, 14, 14, 512)*	*2,359,808*
batch_normalization_15 (BatchNormalization)	*(None, 14, 14, 512)*	*2,048*
activation_1 (Activation)	*(None, 14, 14, 512)*	*0*
Inception_1 (InceptionModule)	*(None, 14, 14, 256)*	*208,928*
Inception_2 (InceptionModule)	*(None, 14, 14, 256)*	*151,584*
Pool4 (GlobalAveragePooling2D)	*(None, 256)*	*0*
predictions (Dense)	*(None, 4)*	*1,028*

۱–۴–۲ نتایج و ارزیابی

ابتدا با استفاده از ابر پارامترهای مدل AlexNet مدل آموزش داده شد، اما در این حالت دقت آن نهایتا به عدد ۶۵ درصد رسید.

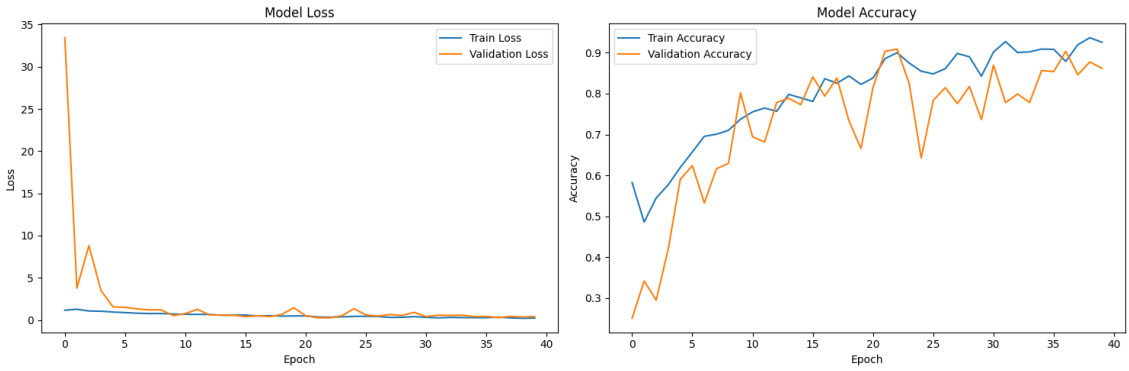
به همین علت ابرپارامترهای دیگری امتحان شدند که ابرپارامتر های جدول ۵ بهترین نتیجه را داشتند.

جدول ۵: پارامترهای مدل

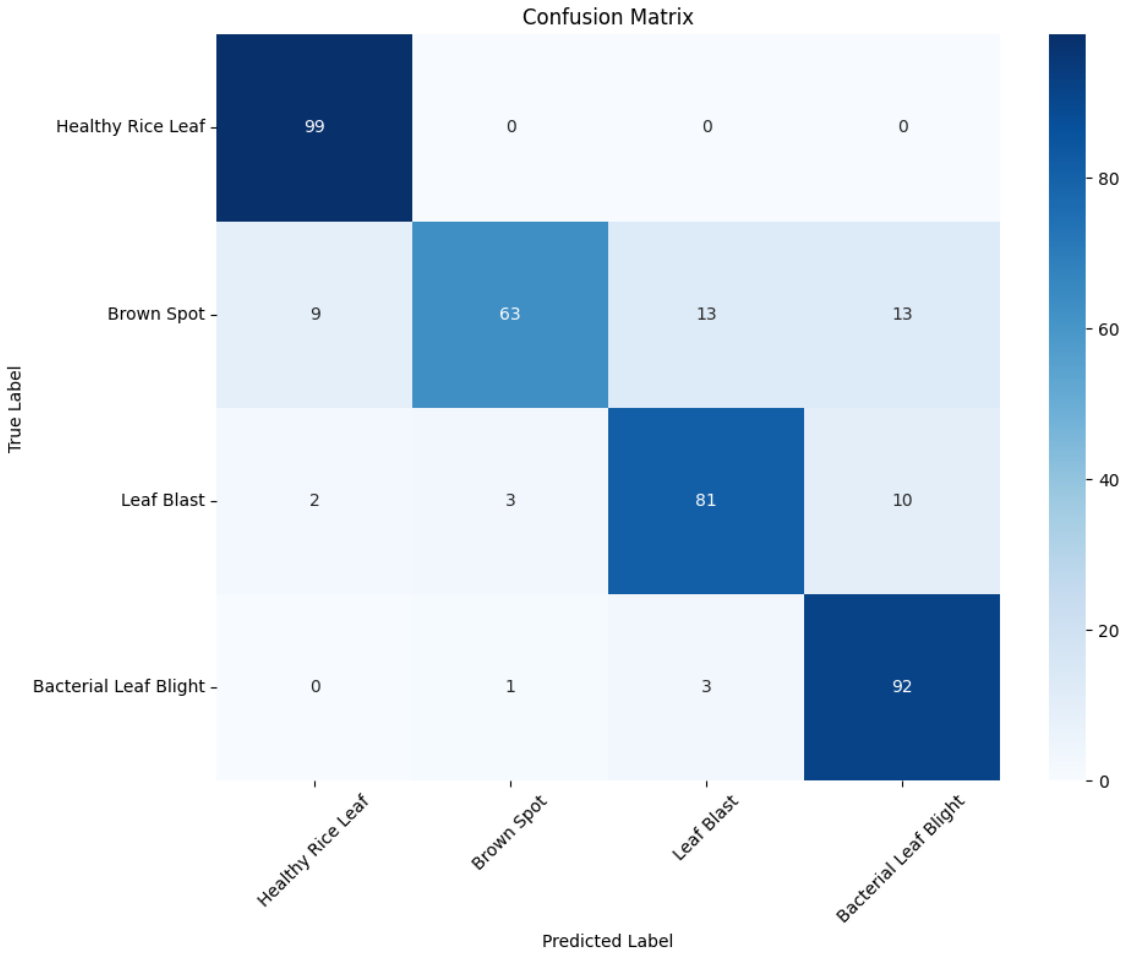
ابرپارامتر	مقدار
Batch Size	*64*
Learning Rate	*0.001*
Optimizer	*Adam*

همچنین شکل ۶ نشان دهنده نمودار تغییرات خطا و دقت آن و شکل ۷ نشان دهند ماتریس درهم ریختگی آن هستند.

در انتها در جدول ۶ سایر معیارهای ارزیابی آن آمده اند.



شکل ۶: نمودار تغییرات خطا و دقت مدل یادگیری انتقالی



شکل ۷: ماتریس درهم ریختگی مدل یادگیری انتقالی

جدول ۶: معیارهای ارزیابی مدل یادگیری انتقالی

Class	*Accuracy*	*Sensitivity*	*Specificity*
Healthy Rice Leaf	*0.9717*	*1.0000*	*0.9621*
Brown Spot	*0.8997*	*0.6429*	*0.9863*
Leaf Blast	*0.9203*	*0.8438*	*0.9454*
Bacterial Leaf Blight	*0.9306*	*0.9583*	*0.9215*

در این مدل دو لایه آخر با دو لایه Inception جایگزین شده اند

پاسخ پرسش‌های مطرح شده در انتهای سوال :

- همانطور که در شکل ۶ آمده است، مدل یادگیری انتقالی سرعت یادگیری بسیار بالاتری از مدل مبتنی بر AlexNet دارد.

- استفاده از وزن های پیش آموخته سبب شده وزن های فعلی بتوانند خیلی از ویژگی های پایه بینایی ماشین مانند استخراج لبه ها را از پیش بلد باشند.

  دانستن این مورد به یادگیری هرچه سریعتر دیتاست جدید کمک کرده است.

- هر ماژول Inception شامل شاخه‌هایی با فیلترهای مختلف (۱×۱، ۳×۳، ۵×۵) است که ویژگی‌ها را در چند مقیاس مختلف استخراج می‌کند.

  با ترکیب شاخه‌ها، مدل توانایی شناسایی الگوهای پیچیده و چندمقیاسی را پیدا می‌کند، بدون اینکه تعداد پارامترها خیلی زیاد شود. اثر جزئیات داده‌ها نیز از بین می رود و کلیات جای آن را می گیرند. همچنین به علت کاهش دادن تعداد پارامترهای مدل سرعت آن را بالا برده است.

- در خیلی از معماری ها، انتهای مدل به یک شبکه کاملا متصل اختصاص یافته تا با استفاده از ویژگی های استخراج شده در مراحل قبل بتواند یک مسئله کلسیفیکیشن را حل کند.

  دلیل استفاده از *Global Pooling* در انتهای معماری ساده تر کردن شبکه می باشد. این لایه به تنهایی تعداد زیاد ویژگی استخراج شده را به تعداد خیلی کمتر کاهش می دهد. این کار سبب می شود یادگیری در لایه های بعدی به سرعت بیشتر و سادگی خیلی بالاتر انجام شود.

  همچنین با کاهش پارامترها سبب می شود تا از مشکلاتی مانند overfitting هم جلوگیری شود. در آخر استفاده از مموری را نیز به علت کم کردن پارامترها کاهش می دهد.

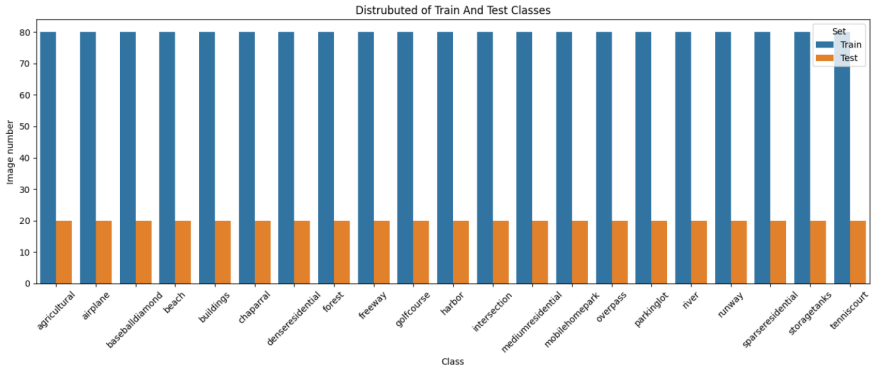
۲ طبقه بندی صحنه در تصاویر سنجش از دور

در این سوال سعی بر ساخت یک مدل cnn و mlp همراه با هم شده است و وضعیت آن و ویژگی های آن در مقایسه با مدل xception مورد برسی قرار گرفته است. کد های این بخش با پیشوند ۲q قرار گرفته است. درواقع تمامی بخش ها به جز دو بخش در فایل $q۲_۱.ipynb$ میباشد. در زدن کدهای این سوال از مدل های زبانی کمک گرفته شده است.همچنین مدل روی کولب و ککل ترین شده است.

۲–۱ آماده سازی مجموعه داده

در این بخش داده های دیتاست دانلود شده و به نسبت ۸۰ به ۲۰ به داده آموزش و تست تقسیم شده است.

همچنین نسبت داده های همه کلاس ها با هم برابر است تعداد هرکدام در دیتاست آموزش و تست و کل دیتاست باهم برابر هستند. در شکل ۸ هیستوگرام توزیع داده ها نمایش داده شده است.



شکل ۸: توزیع کلاس‌های مختلف داده‌ها

۲–۲ پیش پردازش داده ها

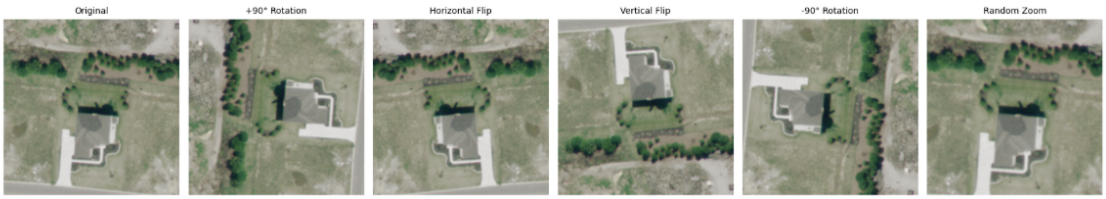
در این بخش پیش پردازش داده ها صورت گرفته است. ابتدا موقع لود دیتاست داده ها به سایز خواسته شده تبدیل شده اند. .

بعد از آن لازم است تا توابعی برای افزایش داده به صورت های گفته شده پیاده سازی کرد. برای این کار این توابع موقع استفاده از دیتاست بر روی عکس فراخوانی می شوند تا از مصرف زیادی مموری جلوگیری

شود. همچنین از توابع تغیر عکس برای خود کتابخوانه pytorch ابخاطر سرعت بیشتر استفاده میکنیم.

دلیل انتخاب این روش ها برای افزایش داده برای این نوع دیتاست بخاطر این است که در عکس های هوایی به صورت طبیعی ممکن است تصویر خیابان ها و هرجای دیگر با هر زاویه و جهتی گرفته شود. با توجه به ارتفاع پرواز هواپیما ممکن است مکان نزدیک تر یا دور تر شده باشد در نتیجه باعث تغیر در تصویر می شود.

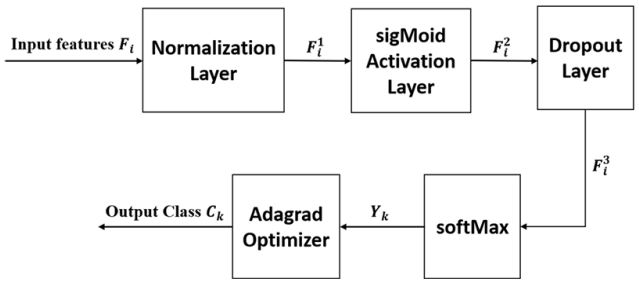
در شکل ۹ نمونه از افزایش داده در کنار تصویر اصلی نمایش داده شده است.



شکل ۹: نمونه تغیر تصویر برای افزایش داده

۲–۳ پیاده سازی

در این بخش مدل از پیش آموزش دیده شده xception لود شده و لایه آخر آن که برای اکلاس بندی استفاده میشده است غیر فعال شده است. همچنین باقی پارامتر های این مدل فریز شده است و برای بخش کلاس بندی از معماری گفته شده در مقاله استفاده شده است. . در شکل ۱۰ معماری بخش کلاس بندی پیشنهادی مدل ارائه شده است.



شکل ۱۰: معماری بخش کلاس بندی مدل ارائه شده در مقاله

در معماری پیشنهادی، از شبکهٔ Xception که پیش‌تر روی مجموعه‌دادهٔ ImageNet آموزش دیده، به‌عنوان استخراج‌کنندهٔ ویژگی استفاده می‌شود. ابتدا تصاویر وارد لایه‌های اولیهٔ شبکه شامل کانولوشن، تابع

فعال‌ساز ReLU و مکس‌پولینگ می‌شوند تا مجموعه‌ای از نقشه‌های ویژگی تولید گردد. سپس لایه‌های میانی مبتنی بر کانولوشن‌های تفکیک‌پذیر عمقی به‌کار گرفته می‌شوند تا ویژگی‌های محلی تصویر استخراج شوند. این لایه‌ها ضمن کاهش پیچیدگی محاسباتی، امکان جداسازی فیلترهای مکانی و عمقی را فراهم کرده و ویژگی‌های محلی را در هر کانال محاسبه می‌کنند. خروجی این بخش به‌صورت یک بردار یک‌بعدی با استفاده از لایهٔ Flatten تبدیل می‌شود. پس از آن، شبکه برای استخراج ویژگی‌های سطح بالا، مجدداً از لایه‌های کانولوشن، ReLU و مکس‌پولینگ استفاده می‌کند، و در نهایت لایهٔ Pooling Average Global ویژگی‌های جهانی را تولید می‌کند که نمایندهٔ کل تصویر است. این ویژگی‌های جهانی سپس به طبقه‌بند نهایی مدل یعنی شبکهٔ MLP داده می‌شوند تا تصمیم‌گیری نهایی در مورد کلاس صحنه انجام گیرد.

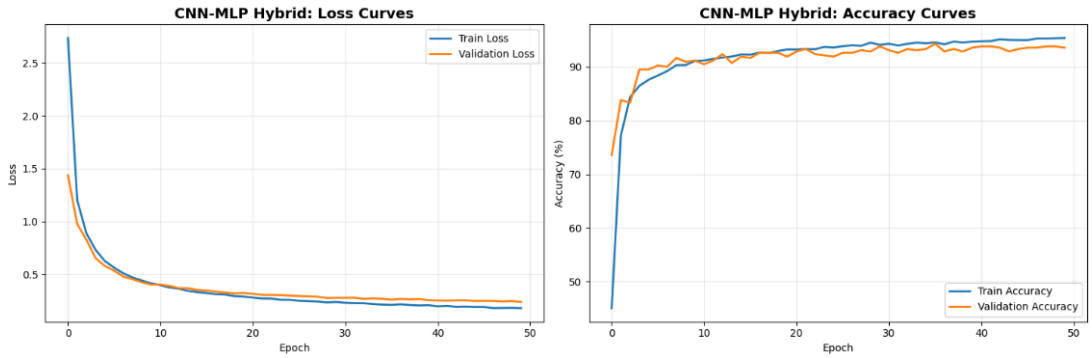
برای ۵۰ epoch این مدل آموزش دیده است. که نرخ جدول دقت آن به ازای هایپر پارامترهای متفاوت در ۷ آمده است. که همانطور که مشاهده می شود نرخ یادگیری ۰۱.۰ مناسب ترین نرخ برای این مدل هست. همچنین با کاهش سایز بچ به ۱۲۸ مقداری دقت مدل افزایش یافته است.

جدول ۷: دقت مدل به ازای هایپرپارامتر های متفاوت

	$rl = 0.01$	$rl = 0.005$	$rl = 0.02$
batch size=200	*93.80*	*92.02*	*93.25*
batch size=128	*94.29*	*92.86*	*94.29*

۲–۴ ارزیابی

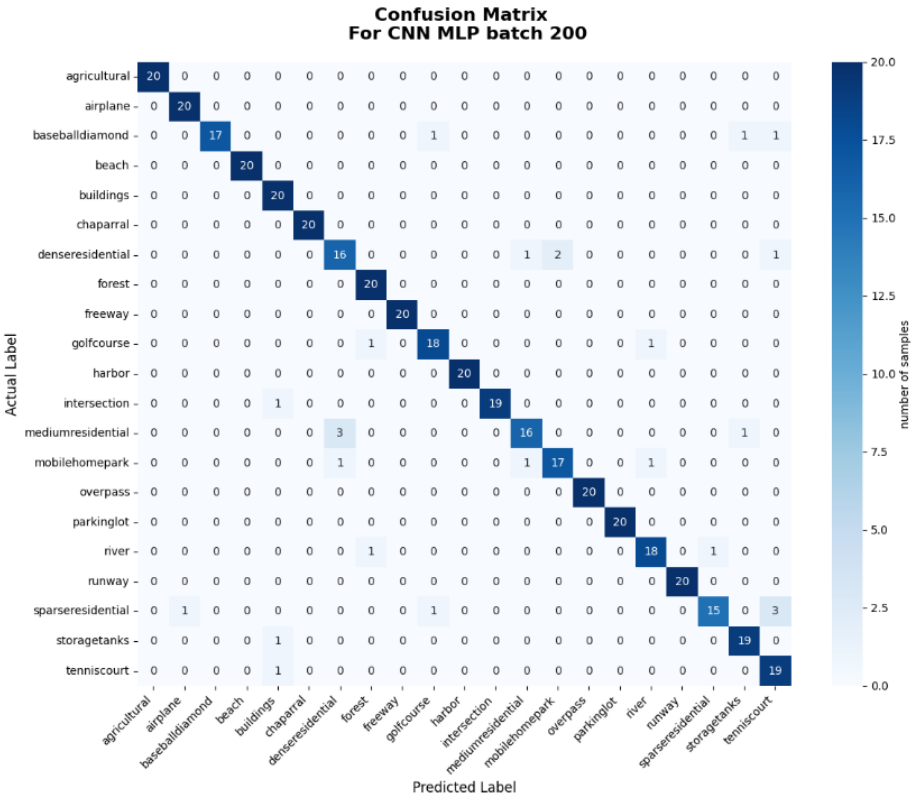
در شکل ۱۱ نمودار خطا و دقت در طول آموزش نشان داده شده است.



شکل ۱۱: نمودار خطا و دقت در آموزش و تست

همانطور که مشاهده می شود دقت مدل به حدود ۹۴ در داده تست رسیده است.

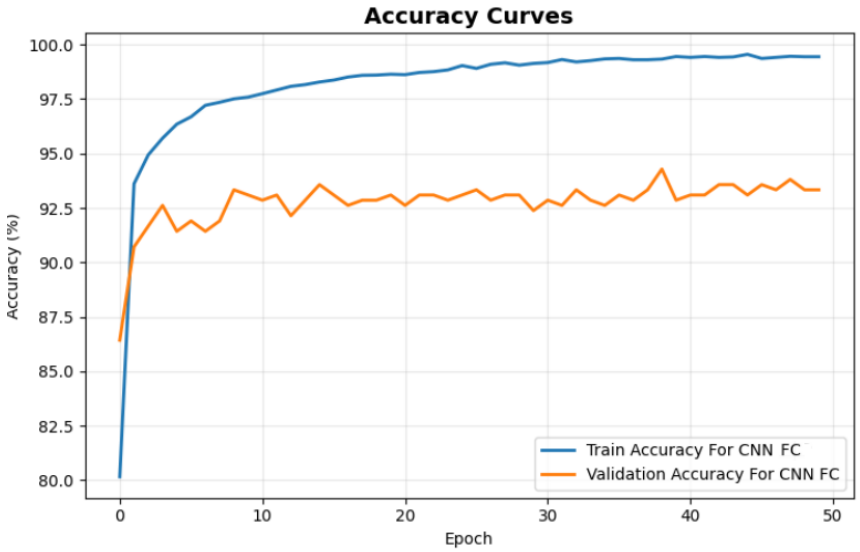
در جدول ۱۲ ماتریس درهم ریختگی برای داده های آزمون برای مدل مقاله نشان داده شده است. که همانطور در آن مشخص است ۱۱ کلاس به طور کامل درست تشخیص داده شده اند. و کلاسی که کمترین تشخیص را داشته است کلاس مناطق مسکونی پراکنده بوده است. که ۳ بار به اشتباه زمین تنیس تشخیص داده شده است. همچنین مناطق مسکونی متوسط ۳ بار به اشتباه مناطق مسکونی متراکم شناخته شده است.



شکل ۱۲: ماتریس درهم ریختگی مدل mlp cnn برای داده تست

با توجه به نمودار آموزش و تست نشان داده شده به نظر می رسد معماری توانسته است بسیار خوب سرعت آموزش را بیشتر کند.

در شکل ۱۳ نمودار دقت خود مدل xception که لایه کلاس بندی آن آموزش دیده شده است نشان داده شده است که همانطور که مشاهده می شود. مدل زود به دقت ۱۰۰ در داده آموزش رسیده است و اتا حدی اورفیت شده است. باقی نمودار ها در فایل نوتبوک ارائه شده آمده است.



شکل ۱۳ : نمودار دقت برای تست و آموزش در مدل fn cnn

همانطور که مشاهده می شود دلیل انتخاب یک لایه جداگانه بجای لایه استفاده شده در xception برای دسته بندی کلاس ها برای جلوگیری از اورفیت و همچنین انعطاف پذیری بیشتر بوده است زیرا خود کلاسیفایر xception برای ۱۰۰۰ کلاس آموزش دیده است.

دلیل لایه pooling Average Global قبل از ورود ویژگی ها به نظر من در ابتدا بخاطر استخراج ویژگی های مهم فارغ از بحث ابعاد تصویر و تبدیل ویژگی ها به یک بردار یک بعدی برای کلاس بندی میباشد.. که درواقع با این کار ویژگی های کلی و مهم تصویر به عنوان ورودی برای کلاس بندی داده می شوند.

در داخل خود مقاله دلیل استفاده از AdaGrad را دقت بهتر وقتی که همراه با یک mlp باشد گفته است. که نرخ یادگیری تطبیقی این بهینه ساز می تواند به دقت بهتر کمک کند. همچنین باعث همگرایی سریع تر مدل می شود.

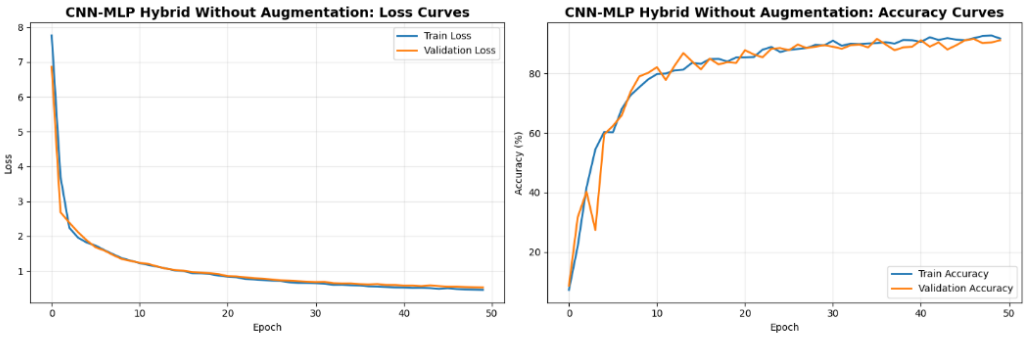
۲–۵ تحلیل و بهینه سازی

۲–۵–۱ تاثیر داده افزایی

در شکل ۱۴ نمودار برای داده آموزش و تست مدل مقاله با دیتاست بدون افزایش داده آورده شده است.هانطور که مشاهده می شود دقت مدل به مراتب پایین تر ازداده همراه با آگمنتیشن بوده است. در

اینجا دقت روی داده های تست حدود ۹۱/۶۶ بوده است

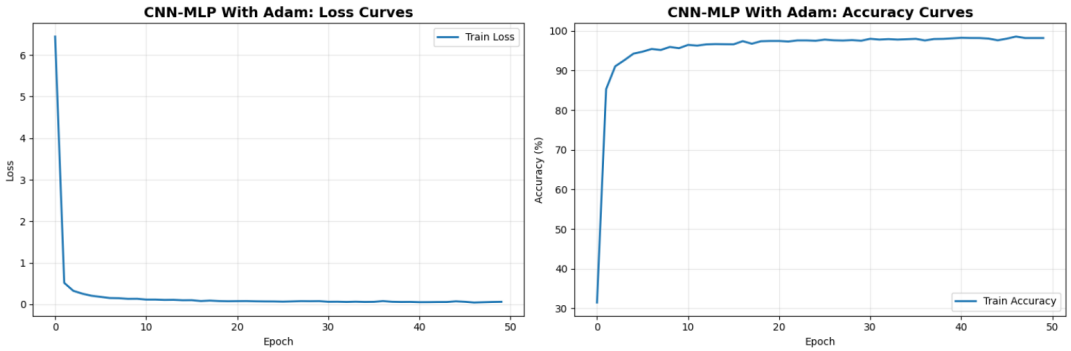
فایل حاوی کدها و دیگر گزارشات مربوط به این بخش در فایل $q۲_۳.ipynb$ آورده شده است.



شکل ۱۴: نمودار دقت و خطا برای مدل بدون افزایش داده

۲–۵–۲ نقش بهینه ساز

در این بخش همان مدل با بهینه ساز Adam آموزش داده شده است دقتی که مدل با بهینه ساز آدام بدست آورده است روی داده تست حدود ۹۵ درصد می باشد که بهتر از مدل آموزش دیده شده با بهینه ساز داخل مقاله است اما با استفاده از این بهینه ساز دقت در داده آموزش به نزدیک ۱۰۰ درصد رسیده است . که نزدیک به خطر اورفیت کردن در داده است. به طورکلی به نظر استفاده از بهینه ساز گفته شده در مقاله در تعداد epoch بیشتر باعث بهبود دقت در مدل می شود. نمودار فرآیند دقت و خطا در آموزش در شکل ۱۵ آمده است. گزارش دقت آن در فایل نوتبوک موجود می باشد.

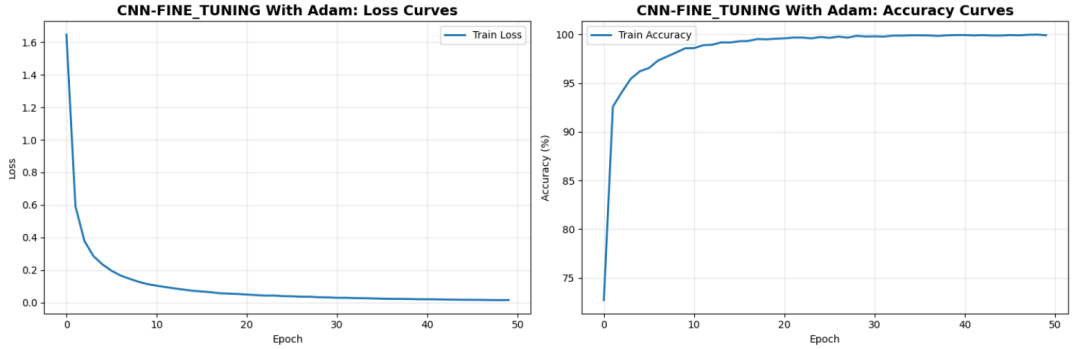


شکل ۱۵: توزیع کلاس‌های مختلف داده‌ها

۲–۵–۳ بهینه سازی توسط fine-tuning

در این بخش حدود ۱۰ لایه آخر مدل xception را فعال می کنیم و با یک نرخ یادگیری کم آن ها را آموزش میدهیم تا به اصطلاح مدل xception گفته شده را فاین تیونگ کنیم کد این بخش در فایل $q۲_۲.ipynb$ آمده است که به دلیل مشکلات ککل نشد تمام خروجی های بخش ها را در یک فایل نوتبوک آماده کنیم. در شکل ۱۶ نمودار های هنگام آموزش برای مدل فاین تیون شده با نرخ یادگیری ۰/۰۰۱ و $batchsize = ۲۰۰$ آورده شده است.

همچنین در جدول ۸ دقت مدل به ازای مقادیر مختلف هایپرپارامتر ها آورده شده است. که با توجه ب آن بهترین نرخ یادگیری برابر با ۰/۰۰۵ می باشد. و با بچ سایز کمتر نیز دقت مدل کمی افزایش میابد.



شکل ۱۶: نمودار خطا و دقت مدل فاین تیون هنگام آموزش

جدول ۸: دقت مدل به ازای هایپرپارامتر های متفاوت در فاین تیونینگ

	$rl = 0.001$	$rl = 0.005$	$rl = 0.002$
batch size=200	*94.52*	*95.12*	*94.25*
batch size=128	*95.00*	*95.62*	*95.48*